

Assignment 6

Retrieval-Augmented Generation (RAG) - Machine Learning Knowledge Assistant

1. Introduction

The objective of this assignment is to build an end-to-end Retrieval-Augmented Generation (RAG) system using a Machine Learning book in PDF format as the knowledge source.

The system is designed to retrieve relevant information from the book and generate accurate, context-based answers for technical questions related to Machine Learning and Deep Learning.

Unlike normal Large Language Models (LLMs), RAG improves answer quality by combining document retrieval with answer generation. This helps reduce hallucination and ensures that responses are grounded in the uploaded document.

2. Dataset Description

The dataset used for this project is a PDF version of a Machine Learning book:

Introduction to Machine Learning with Python

The PDF contains structured knowledge about various Machine Learning and Deep Learning concepts such as:

- Supervised Learning
- Unsupervised Learning
- Neural Networks
- Gradient Descent
- Decision Trees
- Model Evaluation
- Overfitting and Underfitting
- Deep Learning Fundamentals

The PDF acts as the knowledge base for the RAG system.

3. Data Preprocessing

To prepare the PDF for retrieval and answer generation, the following preprocessing steps were performed.

3.1 PDF Loading

The document was loaded using the PyPDFLoader() utility from LangChain.

This process extracts text from each page of the PDF and stores it as structured document objects for further processing.

```
pdf_path = "C:\\Users\\lissac\\Documents\\Next GenAI assignment\\Assignment 6\\introtoml.pdf"

loader = PyPDFLoader(pdf_path)
documents = loader.load()

print("PDF Loaded Successfully")
print("Total Pages:", len(documents))
✓ 7.8s

PDF Loaded Successfully
Total Pages: 392

print(documents[0].page_content[:3000])
✓ 0.0s

Andreas C. Müller & Sarah Guido
Introduction to
Machine
Learning
with P y t h o n
A GUIDE FOR DATA SCIENTISTS
```

3.2 Text Quality Analysis

The extracted content was manually inspected to evaluate:

- text readability
- formatting consistency
- encoding issues
- missing or broken content
- extraction quality

3.3 Text Chunking

Since large documents cannot be processed efficiently as a single text block, the extracted content was divided into smaller overlapping chunks using RecursiveCharacterTextSplitter().

Configuration Used

- Chunk Size: 500
- Chunk Overlap: 50

Purpose of Chunking

Chunking improves retrieval quality by:

- preserving semantic meaning
- reducing information overload
- improving embedding precision
- enabling accurate similarity search

Purpose of Overlap

Overlap preserves continuity between adjacent chunks and prevents the loss of important contextual information across chunk boundaries.

4. Embedding and Vector Database

This stage converts textual content into searchable vector representations and stores them in a vector database for semantic retrieval.

4.1 Embedding Model

The embedding model used in this project is: **sentence-transformers/all-MiniLM-L6-v2**

This model converts each text chunk into dense vector embeddings that capture semantic meaning rather than exact keyword matching.

4.2 Vector Database

The vector database used is:

FAISS (Facebook AI Similarity Search)

FAISS stores all generated embeddings and enables high-speed similarity search.

Advantages of FAISS

- efficient nearest-neighbour search
- optimized vector retrieval
- fast local execution
- scalable architecture for large documents

The vector database was created using:

```
FAISS.from_documents()
```

```
vector_store = FAISS.from_documents(
    texts,
    embedding_model
)

print("FAISS Vector Database Created Successfully")
✓ 1m 6.7s

FAISS Vector Database Created Successfully

vector_store.save_local("faiss_index")

print("Vector DB Saved Successfully")
✓ 0.0s

Vector DB Saved Successfully
```

5. Retrieval Pipeline

The retrieval pipeline identifies the most relevant document chunks for a given user question before answer generation begins.

5.1 User Query Input

The system accepts natural language queries such as:

- What is supervised learning?
- Explain overfitting in machine learning
- What is gradient descent?

This enables intuitive interaction without requiring technical query formatting.

5.2 Query Embedding

The input query is converted into vector form using the same embedding model used for document chunks.

Using a shared embedding model ensures consistency between stored knowledge and incoming questions, improving retrieval relevance.

5.3 Similarity Search

The system retrieves the most relevant chunks using:

```
similarity_search(query, k=3)
```

This performs semantic similarity matching between the user query and stored document embeddings.

5.4 Top-k Retrieval Strategy

The parameter k represents the number of most relevant chunks retrieved before answer generation.

Example

$k = 3$

This means the system retrieves the top three most relevant chunks from the vector database.

6. Answer Generation

After retrieval, the selected document chunks are passed to a Question Answering model for grounded response generation.

6.1 Model Used

Hugging Face Model: **distilbert-base-cased-distilled-squad**

This is an extractive Question Answering model trained specifically for answering questions from provided context.

6.2 Reason for Model Selection

This model was selected because it offers:

- strong performance in question answering tasks
- grounded answer extraction
- lower hallucination risk
- fast inference speed
- reliable local execution without API dependency

6.3 Grounded Answering Strategy

The system follows a three-step process:

1. Retrieve the most relevant chunks
2. Use the retrieved content as answer context
3. Extract the most relevant answer span

Example

Question: Explain overfitting in machine learning

Retrieved Answer: models that are very complex and highly overfit to the training data

Final Improved Answer: Overfitting occurs when a model learns the training data too closely, including noise and unnecessary patterns. This causes the model to perform very well on training data but poorly on unseen data, reducing its ability to generalize effectively.

```
query = "What is supervised learning?"

retrieved_docs = retrieve_docs(query, k=2)

for i, doc in enumerate(retrieved_docs):
    print(f"\n--- Retrieved Chunk {i+1} ---\n")
    print(doc.page_content[:1500])
```

✓ 0.0s Python

--- Retrieved Chunk 1 ---

spam.

Machine learning algorithms that learn from input/output pairs are called supervised learning algorithms because a "teacher" provides supervision to the algorithms in the form of the desired outputs for each example that they learn from. While creating a dataset of inputs and outputs is often a laborious manual process, supervised learning algorithms are well understood and their performance is easy to measure. If your

--- Retrieved Chunk 2 ---

46 | Chapter 2: Supervised Learning

```

questions = [
    "What is supervised learning?",
    "What is overfitting?",
    "Explain neural networks",
    "What is gradient descent?"
]

for q in questions:
    print("\n=====")
    print("QUESTION:", q)
    print("=====")

    print(ask_rag(q, k=5))
    print("\n")

```

[26] ✓ 1.5s

Answer:
Machine learning algorithms that learn from input/output pairs

Context-Based Explanation:
spam.
Machine learning algorithms that learn from input/output pairs are called supervised learning algorithms because a "teacher" provides supervision to the algorithms in the form of the desired outputs for each example that they learn from. While creating a dataset of inputs and outputs is often a laborious manual process, supervised learning algorithms are well understood and their performance is easy to measure. If your

46 | Chapter 2: Supervised Learning

26 | Chapter 2: Supervised Learning

=====

QUESTION: What is overfitting?

=====

Answer:
Choosing
too simple a model

Context-Based Explanation:
occurs when you fit a model too closely to the particularities of the training set and obtain a model that works well on the training set but is not able to generalize to new

```

=====
QUESTION: Explain neural networks
=====

Answer:
the number of
layers and nodes per layer

Context-Based Explanation:
together.
Neural Networks (Deep Learning)
A family of algorithms known as neural networks has recently seen a revival under the name "deep learning." While deep learning shows great promise in many machine learning applications, deep learning algorithms are often tailored very carefully to a specific use case. Here, we will only discuss some relatively simple methods, namely multilayer perceptrons for classification and regression, that can serve as a starting

While it is possible to analyze w

=====
QUESTION: What is gradient descent?
=====

```

7. End-to-End Application

The final system was implemented as an interactive real-time Question Answering application.

The application continuously accepts user queries using a loop-based interface and returns grounded answers until the user exits the session.

Example Questions

- What is supervised learning?
- Explain neural networks
- What is gradient descent?
- What is overfitting?
- Explain decision trees

8. Model Evaluation

The system was tested using multiple technical questions extracted from the Machine Learning book.

Evaluation Observations

- relevant chunks were retrieved successfully
- answers remained grounded in source content
- overfitting and supervised learning questions produced accurate results
- increasing k improved answer richness and explanation quality

9. Conclusion

This project successfully implements a Retrieval-Augmented Generation (RAG) system for Machine Learning knowledge assistance.

The system combines:

- document understanding
- semantic embeddings
- vector similarity search
- context-grounded answer generation

to create a reliable domain-specific Question Answering assistant.

Compared to direct LLM-based generation, this approach provides:

- higher answer accuracy
- lower hallucination
- stronger explainability
- reliable document grounding